

HelixSpecifier

HelixSpecifier

Өзінің рәсімін жұмыс көлеміне бейімдеп өлшейтін спецификацияға негізделген әзірлеу.

Қысқаша мазмұны

HelixSpecifier — үш әзірлеу әдістемесінің — SpecKit-тің спецификацияға негізделген жұмыс ағынын, Superpowers-тің TDD тәртібін және GSD-дің мақсаттық өмірлік циклін — бір бейімдемелі ағынға біріктіретін Go қозғалтқышы. Ол әр тапсырманы күш-жігер деңгейіне қарай жіктейді және процесстің көлемін тиісті түрде реттейді.

Қысқаша сипаттама

HelixSpecifier — AI агенттері үшін спецификацияға негізделген әзірлеудің біріктірілген қозғалтқышы. Ол SpecKit, Superpowers және GSD-ді біріктіріп, жұмысты күш-жігер деңгейіне қарай жіктейді, пікірталастарға негізделген спецификациялық кезеңдерді жүргізеді, кодқа қатысты тесттердің ең төменгі қатынасын қадағалайды және әр аяқталған ағыннан үйренеді.

Толық сипаттама

HelixSpecifier — Go тілінде жазылған спецификацияға негізделген әзірлеу (SDD) біріктірілген қозғалтқышы (`digital.vasic.helixspecifier` модулі), ол HelixAgent AI ансамблінің құрамдас бөлігі ретінде әзірленген. Ол әдетте үш бөлек құралда және үш бөлек көзқараста қолданылатын үш әзірлеу тәжірибесін — SpecKit-тің жеті кезеңді SDD процесі (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), Superpowers-тің тестке негізделген тәртібі мен параллельді ішкі агенттердің орындалуын, сондай-ақ GSD-дің мақсаттық өмірлік циклін басқаруын — бір бейімдемелі жұмыс ағынына

біріктіреді. Әр бағана өз қызметін атқара береді; қозғалтқыш осы бағандарды қолмен тігілген құбырдың орнына біртұтас ағын ретінде жұмыс істеуін қамтамасыз етеді.

Оның негізгі идеясы — *бейімдемелі рәсім*: қозғалтқыш келіп түскен жұмысты күш-жігер деңгейіне қарай жіктейді және процесстің көлемін сәйкестендіреді, яғни бір жолдық түзету ауыр рәсімдерден өткізілмейді, ал үлкен мүмкіндіктер қателік сияқты жеңілдікпен өткізілмейді. Осы негізге он қуатты мүмкіндіктер салынған: шектеулі параллель тапсырмалардың орындалуы, машиналық оқылымды “Constitution коды” ретіндегі автоматты ережелерді сақтау, “Nyquist TDD” тест пен кодтың ең төменгі қатынасын қадағалау, спецификацияны жетілдіру үшін көпраундты көп агенттік пікірталастар, бейімдемелі дағдылық деңгейді үйрену, ескі кодты талдау, тарихи үлгілерден болжамды спецификациялар жасау, жобалар арасындағы білімді беру, жұмыс барысындағы рәсімдерді қайта реттеу және семантикалық іздеу мүмкіндігі бар тұрақты спецификациялық жады.

Ол Go модулі ретінде — `go get` арқылы немесе жергілікті ауыстыру директивасы арқылы — әдейі кішірек жасалған қозғалтқыш API арқылы қолданылады: үш бағананы және рәсімдерді өлшегіш пен спецификациялық жадыны тіркеп, жұмыстың күш-жігер деңгейін жіктеп, толық ағынды орындап, сапалық бағаланған нәтиже алынады. Беткі қабаты қарапайым; бірақ оның артындағы үйлестіру күрделі. Helix отбасының басқа бөліктері сияқты, ол бопсалауға қарсы тексеру режимімен әзірленген, нақты кодты қолданбайтын макеттердің орнына нақты кодты тексеруші ішкі орындаушымен жұмыс істейді.

Неліктен әзірледік

Спецификацияға негізделген әзірлеу, қатаң TDD және мақсаттық басқару әдетте үш бөлек тәжірибе және үш бөлек құрал болып табылады. HelixSpecifier AI агенті (HelixAgent) үшін осы үш тәжірибені қолмен тігіп қосудың орнына біртұтас, өзін-өзі өлшейтін жұмыс ағыны ретінде іске қосу үшін әзірленді.

Мазмұны

Неге бұл өзгерістің қозғаушы күші

Ол үдерісті жұмыс көлеміне сай етіп — автоматты түрде реттейді. Командалар әдетте екі жаман шеткі жағдайдың біріне тұтылып қалады: барлығына ауыр рәсімдер (қауіпсіз, бірақ баяу және жасырын наразылық тудырады) немесе

ешқандай рәсімдер жоқ (тез, бірақ тек уақытша). HelixSpecifier осы компромиссті жойып, әр тапсырманың жіктелген күш-жігеріне сай рәсім көлемін анықтап, жұмыс барысында оны қайта реттейді. Бұрын іске асыру мүмкін болмаған қабілет — әр тапсырмаға сай өзінің көлемін реттейтін үдеріс, ал оған қоса, бір агенттің алғашқы болжамы емес, көп раундты, көп агентті пікірталастың нәтижесіне негізделген техникалық сипаттама шешімдері.

Инновациялық жақтары

- **Бейімделгіш рәсімдер** — үдеріс деңгейі нақты уақыттағы сапа көрсеткіштеріне негізделіп, алдын ала белгіленбей, жұмыс барысында реттеледі.
- **Найквист TDD** — тест пен іске асыру арасындағы қатынас қақпасы (кемінде 2 есе), Найквист дискретизациялау теоремасының логикасын қолданады: мінез-құлықты адал түрде қамту үшін оны оның жиілігінен әлдеқайда жоғары деңгейде үлгілеу керек, сондықтан тесттер олар қамтитын кодты асыра өлшеуі тиіс.
- **Пікірталастық архитектура** — көп раундты, көп агентті техникалық сипаттамаларды жетілдіру, мұнда ұстанымдар ұсынылып, бағаланып, біріктіріледі, бір пікірді әлдебір қарсы пікірмен алмастырады.
- **Болжамды техникалық сипаттамалар және жобалар арасындағы білім беру** — қозғалтқыш жинақталған ағындарды талдап, техникалық сипаттамаларды алдын ала болжайды және бір жобадан алынған қиындықпен жеңілген білімді келесі жобаға тасымалдайды.
- **Constitution код түрінде** — жоба ережелері машина оқитын форматта жазылып, қозғалтқышпен бақыланады, тексерушінің сақтығына қалдырылмайды.

Ең үлкен техникалық қиындықтар және оларды қалай шештік

- **Үш әдіснаманы біріктіріп, олардың бір-бірімен соғысуына жол бермеу** — SpecKit, Superpowers және GSD әрқайсысы жұмыс ағынын өздері басқарады деп есептейді. Бұл мәселе ортақ интерфейс арқылы әр бағана тіркеліп, бір ортақ ағындық өмірлік цикл арқылы басқарылатын біріктіру қозғалтқышымен шешілді, сондықтан олар үш соқтығысқан үдеріс емес, біртұтас үдеріс құрайды.
- **Белгілі бір тапсырмаға қанша үдеріс қажет екенін анықтау** — тым жоғары болжаса, барлығы баяулайды; тым төмен болжаса, қауіпті жұмыс тексерілмей жіберіледі. Бұл мәселе жұмыстың көлемін анықтайтын күш-жігер

жіктеуішімен шешілді, ол рәсімдер көлемін жұмыс барысында динамикалық түрде реттейтін рәсімдер масштабалаушысына беріледі.

- **Техникалық сипаттамалардың сапасын адамның әр шешімді бақылауынсыз жоғары деңгейде сақтау** — бір реттік техникалық сипаттамаларды көп раундты, көп агентті пікірталастық жетілдірумен алмастыру арқылы шешілді, мұнда агенттер раундтар бойынша бәсекелес ұстанымдарды бағалайды, ал Найквист TDD қатынастарының сақталуын талап ету арқылы іске асыру тесттерінен озып кетпейді.

Техникалық стегі

- **Go** — қозғалтқыш бір импортталатын бинарлық файл ретінде жеткізіледі, жұмыс уақытында қосымша жүктемесі жоқ; оның параллельділік моделі шектелген параллель тапсырмаларды таратуды және көп агентті пікірталастық раундтарын орындауды жеңілдетіп, оларды орау мәселесіне айналдырмайды.
- **logrus** — қозғалтқыш пен үш бағана бойынша құрылымдық күнделік жазу, сондықтан ағынның шешімдері (жіктеу, рәсімдер өзгерістері, пікірталастық нәтижелері) кейін оқыла алады.
- **SpecKit бағанасы** — техникалық сипаттамаға негізделген дамудың жеті кезеңді үдерісі (Constitution → Сипаттау → Анықтау → Жоспарлау → Тапсырмалар → Талдау → Іске асыру), техникалық сипаттама кодқа айналатын тәртіпті негізді қамтамасыз етеді.
- **Superpowers бағанасы** — параллельді кіші агенттер орындауы бар TDD тәртібі, тест алдымен жазудың қатаңдығын және іске асыруды адал және жылдам ұстайтын таратуды қамтамасыз етеді.
- **GSD бағанасы** — кезеңдер мен өмірлік циклді басқару, ағынға «аяқталды» деген сезім мен кезеңдер бойынша дамуды береді.
- **Техникалық сипаттамалар жадысы** — өткен техникалық сипаттамалардың тұрақты, семантикалық іздеуге болатын индексі, ол болжамды техникалық сипаттамалар мен жобалар арасындағы білім берудің негізі болып, әр жолы суықтан бастаудың орнына қол жетімді етеді.

Мәтін

Статус және адалдық ескертпелері

- **Статус: бета.** HelixAgent модулінің Go құрамы ретінде пайдаланылды.

- **Лицензия: анықталмаған.** GitHub API арқылы ЛИЦЕНЗИЯ табылмады — ТЕКСЕРІЛМЕГЕН / көрсетілмеген.
- “HelixSpecifier” атауы specifier деген репозиторийге сәйкес келеді.

Басымдық деңгейі: Helix-басты.